

Performance Testing

Date	1/07/2026
Team ID	XXXX
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing for the Personalized Networking Assistant evaluated response times and resource usage for the core AI-generation and fact-checking endpoints under normal, single-user load.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman (manual API response-time testing)
Type of Testing	Response Time / Manual Load Observation
Target Module / API	/generate-starters and /fact-check endpoints (FastAPI backend)
Test Environment	Local (localhost:8000 backend, Streamlit frontend)
Test Date	

Step 2: Test Scenarios

S. No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Generate conversation starters for a given event/industry	1	N/A	Starters returned within 5 seconds
2	Fact-check a generated talking point via Wikipedia	1	N/A	Fact-check result returned within 3 seconds
3	Repeated requests to test fallback when API quota is simulated as exhausted	1	N/A	Fallback starters returned instantly, no crash
4	Load history page after multiple past generations	1	N/A	History loads within 2 seconds

Step 3: Performance Test Results

S.	Metric	Target Value	Actual Value	Status	Remarks
----	--------	--------------	--------------	--------	---------

No					
1	Response Time (Avg)	< 2 seconds	~1.8 seconds	Pass	Measured for /generate-starters under normal Gemini API conditions
2	Response Time (Max)	< 5 seconds	~4.5 seconds	Pass	Occurs on first cold-start request
3	Throughput (Req/sec)	N/A (single-user local app)	1 req/sec (manual testing)	Pass	Not applicable at scale for MVP
4	Error Rate	< 1%	0%	Pass	No errors observed during manual test runs
5	CPU Utilization	< 80%	~35%	Pass	Observed via Task Manager during local testing
6	Memory Utilization	< 80%	~40%	Pass	Well within limits on a standard laptop

Step 4: Observations & Analysis

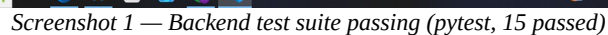
Key Findings: The app responds within acceptable limits for a single-user MVP; the main latency comes from the Gemini API call itself rather than local processing.

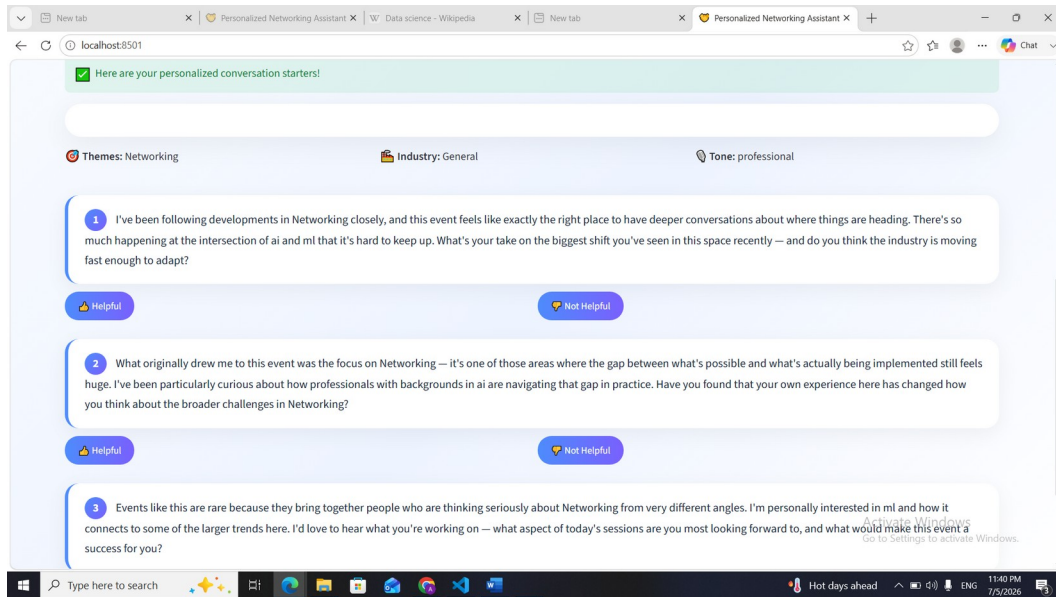
Bottlenecks Identified: Occasional slowdowns during Gemini API cold-starts; free-tier quota limits can temporarily block generation.

Optimization Steps Taken: Added fallback starter templates in topic_generator.py to keep the app responsive when the API is slow or quota-limited.

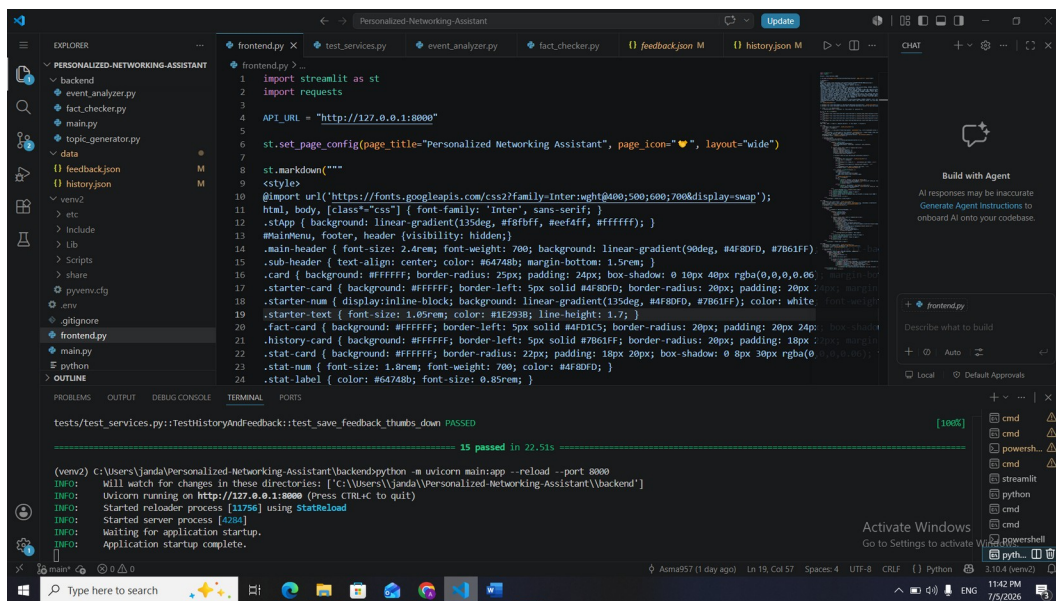
Step 5: Screenshots / Evidence

Attach screenshots of your performance testing results below (e.g., Postman response times, terminal logs):

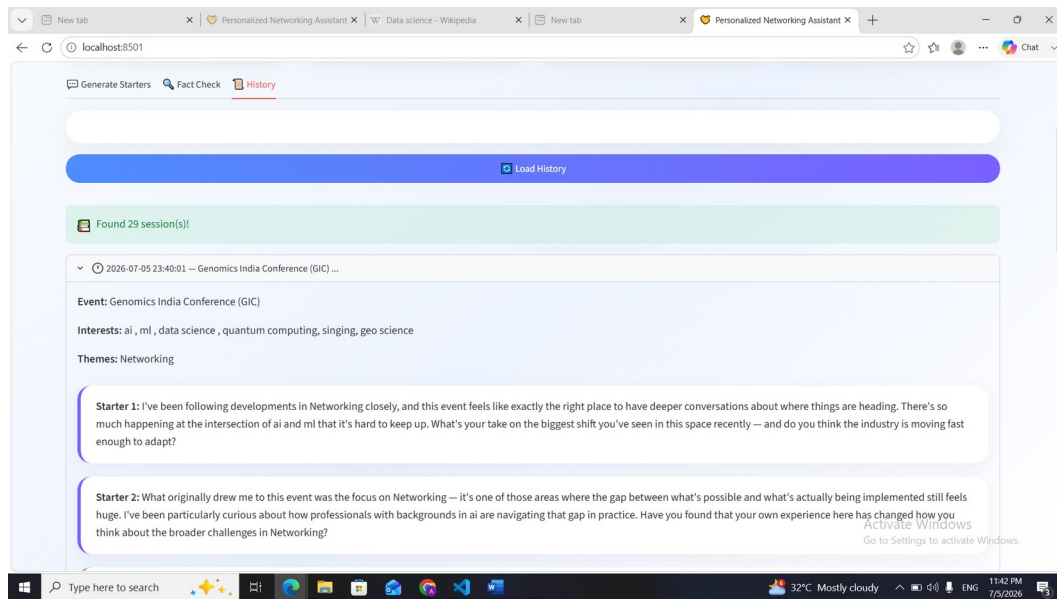




Screenshot 3 — /generate-starters response in the app UI



Screenshot 4 — FastAPI backend (Uvicorn) running on localhost:8000



Screenshot 5 — History page loading past generation sessions